

# คู่มือติดตั้ง CareFlow บน Windows 11

## สำหรับผู้เริ่มต้น

**SYNTHETIC UAT ONLY** - ใช้ข้อมูลสังเคราะห์และเครื่องทดสอบภายในเครื่องเท่านั้น

คู่มือนี้ไม่ใช้การ deploy ระบบจริง ห้ามใช้ข้อมูลผู้ป่วยจริง ข้อมูลทางคลินิกจริง หรือข้อมูลการเงินจริง และห้ามเปิดระบบออกสู่เครือข่าย ห้ามใช้ข้อมูลการเงินจริง

### ผลลัพธ์เมื่อทำครบ

- CareFlow เปิดเฉพาะที่ `http://127.0.0.1:3001`
- เข้าสู่ระบบด้วย `uat-assistant` ใน Browser Profile สำหรับผู้ช่วยได้
- เข้าสู่ระบบด้วย `uat-doctor` ใน Browser Profile สำหรับแพทย์ได้
- พร้อมเปิดคู่มือ UAT ทั้ง 5 Scenario โดยใช้ฐานข้อมูลเดิม

### สารบัญย่อ

1. Checkpoint 1 - ตรวจสอบเครื่อง Windows
2. Checkpoint 2 - เลือกโฟลเดอร์ภายในเครื่อง
3. Checkpoint 3 - ดาวน์โหลด CareFlow จาก GitHub
4. Checkpoint 4 - ติดตั้ง dependencies
5. Checkpoint 5 - ตรวจสอบ native runtime
6. Checkpoint 6 - ทดสอบและ build
7. Checkpoint 7 - สร้างพื้นที่ฐานข้อมูลที่จำกัดสิทธิ์
8. Checkpoint 8 - สร้างโครงสร้างฐานข้อมูล
9. Checkpoint 9 - สร้างสองบัญชี UAT
10. Checkpoint 10 - เปิดระบบและเข้าสู่ระบบสองบทบาท

### คำศัพท์ก่อนเริ่ม

- **PowerShell:** หน้าต่างที่ใช้วางคำสั่งบน Windows
- **Repository:** โฟลเดอร์โครงการ CareFlow ที่ดาวน์โหลดจาก GitHub
- **Clone:** การดาวน์โหลด Repository ลงเครื่องด้วย Git
- **Dependency:** โปรแกรมย่อยที่ CareFlow ต้องใช้
- **Migration:** การสร้างโครงสร้างฐานข้อมูล SQLite ตามรุ่นที่ถูกต้อง
- **Host:** โปรแกรม CareFlow ที่กำลังเปิดอยู่ในหน้าต่าง PowerShell
- **Browser Profile:** โปรไฟล์เบราว์เซอร์ที่แยก session ของผู้ช่วยและแพทย์
- **UAT:** การทดลองใช้งานตาม Scenario ก่อน Pilot

ใช้บัญชี Windows UAT เฉพาะที่ไม่ใช่ผู้ดูแล เปิด PowerShell ปกติ ไม่เลือก Run as administrator และใช้หน้าต่าง PowerShell เดิมต่อเนื่องตั้งแต่ Checkpoint 1 ถึง Checkpoint 10

## Checkpoint 1 - ตรวจสอบเครื่อง Windows

### เป้าหมาย

ยืนยันจากค่าที่คำสั่งแสดงว่าเป็น Windows 11 แบบ 64-bit, PowerShell 5.1 ขึ้นไปที่ไม่ยกระดับสิทธิ์, Node.js 22, Git และตำแหน่งติดตั้งอยู่บน local fixed NTFS โดยไม่ผ่าน OneDrive, network/shared location, WSL, symlink หรือ junction

### ทำตามนี้

ใน PowerShell ปกติที่ไม่ได้เลือก Run as administrator ให้วาง block นี้ทั้ง block ตามตัวอักษร คำสั่งนี้ใช้ syntax ที่รองรับ Windows PowerShell 5.1 และจะหยุดแบบ fail-closed เมื่อยืนยันค่าใดไม่ได้:

```
$ErrorActionPreference = 'Stop'
try {
    $os = Get-CimInstance -ClassName Win32_OperatingSystem -ErrorAction Stop
    $windows11 = $os.ProductType -eq 1 -and [int]$os.BuildNumber -ge 22000
    $operatingSystem64Bit = [System.Environment]::Is64BitOperatingSystem
    $powershellProcess64Bit = [System.Environment]::Is64BitProcess
    $powershellVersion = $PSVersionTable.PSVersion
    $powershell510rNewer = $PSVersionTable.PSVersion -ge [version]'5.1'

    $windowsIdentity = [System.Security.Principal.WindowsIdentity]::GetCurrent()
    $windowsPrincipal = [System.Security.Principal.WindowsPrincipal]::new($windowsIdentity)
    $runningAsAdministrator = $windowsPrincipal.IsInRole(
        [System.Security.Principal.WindowsBuiltInRole]::Administrator
    )

    $nodeVersion = 'NOT FOUND'
    $node22 = $false
    try {
        $nodeCommand = (Get-Command -Name node -CommandType Application -ErrorAction Stop).Source
        $nodeOutput = & $nodeCommand --version 2>&1
        $nodeExitCode = $LASTEXITCODE
        $nodeVersion = ($nodeOutput | Out-String).Trim()
        $node22 = $nodeExitCode -eq 0 -and $nodeVersion -match '^v22\.'
    } catch {
        $nodeVersion = 'NOT FOUND'
        $node22 = $false
    }

    $gitVersion = 'NOT FOUND'
    $gitAvailable = $false
    try {
        $gitCommand = (Get-Command -Name git -CommandType Application -ErrorAction Stop).Source
        $gitOutput = & $gitCommand --version 2>&1
        $gitExitCode = $LASTEXITCODE
        $gitVersion = ($gitOutput | Out-String).Trim()
        $gitAvailable = $gitExitCode -eq 0 -and $gitVersion -match '^git version '
    } catch {
        $gitVersion = 'NOT FOUND'
        $gitAvailable = $false
    }

    if ([string]::IsNullOrEmpty($env:LOCALAPPDATA)) {
        throw 'LOCALAPPDATA is unavailable; UAT is BLOCKED'
    }
    $sourceParent = ([System.IO.Path]::GetFullPath($env:LOCALAPPDATA)).TrimEnd('\')
    if (-not (Test-Path -LiteralPath $sourceParent -PathType Container -ErrorAction Stop)) {
        throw 'The LOCALAPPDATA source parent does not exist; UAT is BLOCKED'
    }
    $validatedSourceRoot = [System.IO.Path]::GetFullPath(
        (Join-Path $sourceParent 'CareFlow-UAT-Source')
    )
    $sourcePathRoot = [System.IO.Path]::GetPathRoot($sourceParent)
    $sourceDrive = $sourcePathRoot.TrimEnd('\')

    $sourceParentIsWSL = (
        $sourceParent.StartsWith('\wsl$', [System.StringComparison]::OrdinalIgnoreCase) -or
        $sourceParent.StartsWith('\wsl.localhost\ ', [System.StringComparison]::OrdinalIgnoreCase)
    )
    $sourceParentIsNetworkOrShared = $sourcePathRoot.StartsWith('\')
}
```

```

$volumeDriveType = [System.IO.DriveType]::Unknown
$volumeFileSystem = 'UNKNOWN'
if ($sourceDrive -match '^[A-Za-z]:$') {
    $escapedSourceDrive = $sourceDrive.Replace('"', '\"')
    $logicalDisk = Get-CimInstance -ClassName Win32_LogicalDisk -Filter "DeviceID = '$escapedSourceDrive'"
    -ErrorAction Stop
    if ($null -eq $logicalDisk) {
        throw 'The source volume could not be identified; UAT is BLOCKED'
    }
    $volumeDriveType = [System.IO.DriveType][int]$logicalDisk.DriveType
    $volumeFileSystem = [string]$logicalDisk.FileSystem
} else {
    $sourceParentIsNetworkOrShared = $true
}
$sourceParentIsNetworkOrShared = (
    $sourceParentIsNetworkOrShared -or
    $volumeDriveType -eq [System.IO.DriveType]::Network
)

$sourceParentIsOneDrive = $sourceParent -match '(?i)(^|\\)OneDrive(?:[^\\"]*)?(\\|$)'
foreach ($oneDriveCandidate in @(
    $env:OneDrive,
    $env:OneDriveCommercial,
    $env:OneDriveConsumer
)) {
    if (-not [string]::IsNullOrEmpty($oneDriveCandidate)) {
        $oneDriveRoot = ([System.IO.Path]::GetFullPath($oneDriveCandidate)).TrimEnd('\')
        if (
            $sourceParent.Equals($oneDriveRoot, [System.StringComparison]::OrdinalIgnoreCase) -or
            $sourceParent.StartsWith($oneDriveRoot + '\', [System.StringComparison]::OrdinalIgnoreCase)
        ) {
            $sourceParentIsOneDrive = $true
        }
    }
}

$reparsePointPath = $null
$pathItem = Get-Item -LiteralPath $sourceParent -Force -ErrorAction Stop
while ($null -ne $pathItem) {
    if (($pathItem.Attributes -band [System.IO.FileAttributes]::ReparsePoint) -ne 0) {
        $reparsePointPath = $pathItem.FullName
        break
    }
    $pathItem = $pathItem.Parent
}
$sourceParentHasReparsePoint = $null -ne $reparsePointPath

Write-Output ("WindowsProductName = {0}" -f $os.Caption)
Write-Output ("WindowsBuildNumber = {0}" -f $os.BuildNumber)
Write-Output ("Windows11 = {0}" -f $windows11)
Write-Output ("OperatingSystem64Bit = {0}" -f $operatingSystem64Bit)
Write-Output ("PowerShellProcess64Bit = {0}" -f $powerShellProcess64Bit)
Write-Output ("PowerShellVersion = {0}" -f $powerShellVersion)
Write-Output ("PowerShell510rNewer = {0}" -f $powerShell510rNewer)
Write-Output ("RunningAsAdministrator = {0}" -f $runningAsAdministrator)
Write-Output ("NodeVersion = {0}" -f $nodeVersion)
Write-Output ("Node22 = {0}" -f $node22)
Write-Output ("GitVersion = {0}" -f $gitVersion)
Write-Output ("GitAvailable = {0}" -f $gitAvailable)
Write-Output ("SourceParent = {0}" -f $sourceParent)
Write-Output ("VolumeDriveType = {0}" -f $volumeDriveType)
Write-Output ("VolumeFileSystem = {0}" -f $volumeFileSystem)
Write-Output ("SourceParentIsOneDrive = {0}" -f $sourceParentIsOneDrive)
Write-Output ("SourceParentIsNetworkOrShared = {0}" -f $sourceParentIsNetworkOrShared)
Write-Output ("SourceParentIsWsl = {0}" -f $sourceParentIsWsl)
Write-Output ("SourceParentHasReparsePoint = {0}" -f $sourceParentHasReparsePoint)
Write-Output ("ValidatedSourceRoot = {0}" -f $validatedSourceRoot)

if (-not $windows11) { throw 'Windows 11 is required; UAT is BLOCKED' }
if (-not $operatingSystem64Bit -or -not $powerShellProcess64Bit) { throw '64-bit Windows and PowerShell are required; UAT is BLOCKED' }
if (-not $powerShell510rNewer) { throw 'PowerShell 5.1 or newer is required; UAT is BLOCKED' }
if ($runningAsAdministrator) { throw 'PowerShell must not run as Administrator; UAT is BLOCKED' }
if (-not $node22) { throw 'Node.js 22 is required; UAT is BLOCKED' }

```

```

if (-not $gitAvailable) { throw 'Git is required; UAT is BLOCKED' }
if ($volumeDriveType -ne [System.IO.DriveType]::Fixed -or $volumeFileSystem -ne 'NTFS') { throw 'The
source parent must be on a local fixed NTFS volume; UAT is BLOCKED' }
if ($sourceParentIsOneDrive) { throw 'The source parent must not be inside OneDrive; UAT is BLOCKED' }
if ($sourceParentIsNetworkOrShared) { throw 'The source parent must not be a network or shared location;
UAT is BLOCKED' }
if ($sourceParentIsWsl) { throw 'The source parent must not be inside WSL; UAT is BLOCKED' }
if ($sourceParentHasReparsePoint) { throw 'The source parent must not use a symlink or junction; UAT is
BLOCKED' }
} catch {
if ($_.Exception.Message -like '*UAT is BLOCKED') { throw }
throw ("Preflight could not verify this machine: {0}; UAT is BLOCKED" -f $_.Exception.Message)
}

```

### ผลที่ต้องเห็น

ต้องเห็นข้อความบรรทัดและค่าต่อไปนี้: Windows11 = True, OperatingSystem64Bit = True, PowerShellProcess64Bit = True, PowerShell510rNewer = True, RunningAsAdministrator = False, Node22 = True, GitAvailable = True, VolumeDriveType = Fixed, VolumeFileSystem = NTFS, SourceParentIsOneDrive = False, SourceParentIsNetworkOrShared = False, SourceParentIsWsl = False และ SourceParentHasReparsePoint = False. บรรทัด PowerShellVersion, NodeVersion, GitVersion, SourceParent และ ValidatedSourceRoot ต้องมีค่าจริง โดย ValidatedSourceRoot ลงท้ายด้วย CareFlow-UAT-Source

### ถ้าไม่ตรงให้หยุด

หาก block แสดง UAT is BLOCKED, ค่าใดหายไป หรือค่าใดไม่ตรงรายการข้างต้น ให้หยุดและส่งเฉพาะค่าที่แสดงแก่ผู้ดูแล ห้ามเปลี่ยนไปใช้บัญชีผู้ดูแล, WSL, Docker, path อื่น หรือข้าม guard

## Checkpoint 2 - เลือกโฟลเดอร์ภายในเครื่อง

### เป้าหมาย

รับช่วงตำแหน่ง local fixed NTFS ที่ผ่านการตรวจจาก Checkpoint 1 และยืนยันว่า source folder ใหม่ยังไม่มีอยู่

### ทำตามนี้

ใช้ PowerShell หน้าต่างเดิมจาก Checkpoint 1 แล้ววาง block นี้:

```

$sourceRoot = $validatedSourceRoot
if ([string]::IsNullOrEmpty($sourceRoot)) {
throw 'Validated source location is unavailable; UAT is BLOCKED'
}
if (Test-Path -LiteralPath $sourceRoot -PathType Any -ErrorAction Stop) {
throw 'CareFlow source folder already exists; installation is BLOCKED'
}
Write-Output ("SourceRoot = {0}" -f $sourceRoot)
Write-Output 'SourceFolderAvailable = True'

```

### ผลที่ต้องเห็น

เห็น SourceRoot ตรงกับ ValidatedSourceRoot จาก Checkpoint 1 และเห็น SourceFolderAvailable = True

### ถ้าไม่ตรงให้หยุด

หากตัวแปรหายไป, path เปลี่ยนไป, พบโฟลเดอร์เดิม หรือเห็น UAT is BLOCKED ให้หยุด ห้ามสร้าง ลบ หรือย้ายของเดิม และแจ้งผู้ดูแล

## Checkpoint 3 - ดาวน์โหลด CareFlow จาก GitHub

### เป้าหมาย

ได้ checkout ใหม่บน branch main

## ทำตามนี้

```
git clone https://github.com/ajbk/careflow-clinic.git "$sourceRoot"
Set-Location -LiteralPath $sourceRoot -ErrorAction Stop
git switch main
git pull --ff-only
git status --short --branch
```

## ผลที่ต้องเห็น

คำสั่งสุดท้ายแสดง main โดยไม่มี error หรือไฟล์แก้ไขค้าง

## ถ้าไม่ตรงให้หยุด

เมื่อเห็น BLOCKED, clone ล้มเหลว, branch ไม่ใช่ main หรือมีไฟล์ค้าง ให้หยุดและแจ้งผู้ดูแล.

## Checkpoint 4 - ติดตั้ง dependencies

### เป้าหมาย

เข้าสู่ careflow-pilot ที่ตรวจสอบแล้วและติดตั้ง packages โดยไม่รัน lifecycle scripts

## ทำตามนี้

วาง block นี้ตามตัวอักษร:

```
$startingDirectory = [System.IO.Path]::GetFullPath((Get-Location).Path)
$pilotRoot = if ([System.IO.Path]::GetFileName($startingDirectory) -eq 'careflow-pilot') {
    $startingDirectory
} else {
    [System.IO.Path]::GetFullPath((Join-Path $startingDirectory 'careflow-pilot'))
}
$pilotManifest = [System.IO.Path]::GetFullPath((Join-Path $pilotRoot 'package.json'))
if (-not (Test-Path -LiteralPath $pilotManifest -PathType Leaf -ErrorAction Stop)) {
    throw 'CareFlow pilot root not found; UAT is BLOCKED'
}
$pilotPackage = Get-Content -LiteralPath $pilotManifest -Raw -ErrorAction Stop | ConvertFrom-Json
if ($pilotPackage.name -ne 'careflow-pilot') {
    throw 'CareFlow pilot root is invalid; UAT is BLOCKED'
}
Set-Location -LiteralPath $pilotRoot -ErrorAction Stop
npm ci --ignore-scripts
```

## ผลที่ต้องเห็น

ติดตั้งจบโดยไม่มี error และ prompt อยู่ใน careflow-pilot

## ถ้าไม่ตรงให้หยุด

ทำเครื่องหมาย UAT เป็น BLOCKED; ห้ามใช้ npm ci แบบอื่นหรือติดตั้ง compiler.

## Checkpoint 5 - ตรวจ native runtime

### เป้าหมาย

ตรวจว่า native packages ทำงานบน Windows เครื่องนี้

## ทำตามนี้

```
npm run verify:native-runtime
```

## ผลที่ต้องเห็น

คำสั่งจบโดยไม่มี error

## ถ้าไม่ตรงให้หยุด

ทำเครื่องหมาย UAT เป็น BLOCKED; ห้ามติดตั้ง Visual Studio, node-gyp หรือ compiler.

## Checkpoint 6 - ทดสอบและ build

### เป้าหมาย

ตรวจสอบ source ก่อนสร้างข้อมูล UAT

### ทำตามนี้

```
npm test
npm run lint
npm run typecheck
npm run build
```

### ผลที่ต้องเห็น

ทั้งสัปดาห์จบโดยไม่มี failure หรือ error

## ถ้าไม่ตรงให้หยุด

ทำเครื่องหมาย UAT เป็น BLOCKED; ห้ามแก้ไข source, migration หรือ workflow ระหว่างการติดตั้ง.

## Checkpoint 7 - สร้างพื้นฐานข้อมูลที่จำกัดสิทธิ์

### เป้าหมาย

สร้าง data\uat ใหม่ด้วย DACL ที่มีเพียงบัญชี UAT ปัจจุบันและ Administrators

### ทำตามนี้

ใน PowerShell เดิม วาง block นี้ตามตัวอักษร:

```
$uatDir = [System.IO.Path]::GetFullPath((Join-Path $pilotRoot 'data\uat'))
$uatDb = [System.IO.Path]::GetFullPath((Join-Path $uatDir 'careflow-uat.sqlite'))
$uatParent = [System.IO.Path]::GetDirectoryName($uatDir)
New-Item -ItemType Directory -Path $uatParent -Force -ErrorAction Stop | Out-Null
if (Test-Path -LiteralPath $uatDir -PathType Any -ErrorAction Stop) { throw 'Use a new UAT directory' }
New-Item -ItemType Directory -Path $uatDir -ErrorAction Stop | Out-Null
$currentSid = [System.Security.Principal.WindowsIdentity]::GetCurrent().User.Value
$administratorsSid = 'S-1-5-32-544'
$expectedSids = @($currentSid, $administratorsSid)
$inheritanceFlags = [System.Security.AccessControl.InheritanceFlags]::ContainerInherit -bor
[System.Security.AccessControl.InheritanceFlags]::ObjectInherit
$uatAcl = [System.Security.AccessControl.DirectorySecurity]::new()
$uatAcl.SetAccessRuleProtection($true, $false)
foreach ($sid in $expectedSids) {
    $identity = [System.Security.Principal.SecurityIdentifier]::new($sid)
    $rule = [System.Security.AccessControl.FileSystemAccessRule]::new(
        $identity,
        [System.Security.AccessControl.FileSystemRights]::FullControl,
        $inheritanceFlags,
        [System.Security.AccessControl.PropagationFlags]::None,
        [System.Security.AccessControl.AccessControlType]::Allow
    )
    [void]$uatAcl.AddAccessRule($rule)
}
Set-Acl -LiteralPath $uatDir -AclObject $uatAcl -ErrorAction Stop

$actualAcl = Get-Acl -LiteralPath $uatDir -ErrorAction Stop
$actualRules = @($actualAcl.Access)
$invalidRules = @($actualRules | Where-Object {
    $ruleSid = $_.IdentityReference.Translate([System.Security.Principal.SecurityIdentifier]).Value
    $ruleSid -notin $expectedSids -or
    $_.AccessControlType -ne [System.Security.AccessControl.AccessControlType]::Allow -or
    $_.FileSystemRights -ne [System.Security.AccessControl.FileSystemRights]::FullControl -or
    $_.InheritanceFlags -ne $inheritanceFlags -or
    $_.PropagationFlags -ne [System.Security.AccessControl.PropagationFlags]::None -or
```

```

    $_.IsInherited
  })
  $missingSids = @($expectedSids | Where-Object {
    $expectedSid = $_
    -not ($actualRules | Where-Object {
      $_.IdentityReference.Translate([System.Security.Principal.SecurityIdentifier]).Value -eq $expectedSid
    })
  })
  if (-not $actualAcl.AreAccessRulesProtected -or
    $actualRules.Count -ne $expectedSids.Count -or
    $invalidRules.Count -ne 0 -or
    $missingSids.Count -ne 0) {
    throw 'UAT directory ACL validation failed; UAT is BLOCKED'
  }
  & iccls.exe $uatDir
  if ($LASTEXITCODE -ne 0) { throw 'UAT directory ACL display failed; UAT is BLOCKED' }

```

### ผลที่มองเห็น

ไม่มี error และ iccls.exe แสดงเฉพาะ trustee ที่กำหนด

### ถ้าไม่ตรงให้หยุด

หากมีโฟลเดอร์เดิมหรือ ACL validation ล้มเหลว ให้ BLOCKED; ห้ามลบโฟลเดอร์หรือฟอนลิทธี.

## Checkpoint 8 - สร้างโครงสร้างฐานข้อมูล

### เป้าหมาย

migrate ฐานข้อมูลใหม่ที่ \$uatDb เดิม

### ทำตามนี้

วาง block นี้ตามตัวอักษร:

```

$env:CAREFLOW_DB_PATH = $uatDb
$env:CAREFLOW_HOST = '127.0.0.1'
$env:CAREFLOW_PORT = '3001'
$env:CAREFLOW_COOKIE_SECURE = 'false'
npm run db:migrate -- "$uatDb"
if ($LASTEXITCODE -ne 0) { throw 'UAT migration failed' }

```

### ผลที่มองเห็น

migration จบโดยไม่มี error; บันทึก OS, รุ่น Node, \$uatDb และผล iccls.exe ใน checklist โดยไม่บันทึกข้อมูลลับ

### ถ้าไม่ตรงให้หยุด

ทำเครื่องหมาย UAT เป็น BLOCKED; ห้าม reset, เปลี่ยน path หรือใช้ฐานข้อมูลเดิม.

## Checkpoint 9 - สร้างสองบัญชี UAT

### เป้าหมาย

สร้างเพียง uat-assistant และ uat-doctor ก่อนเปิด host

### ทำตามนี้

ใน PowerShell เดิมที่มี CAREFLOW\_DB\_PATH แล้ว รันตามลำดับครั้งละหนึ่งคำสั่ง ยอมรับ SYNTHETIC-ONLY และใช้ hidden terminal input สำหรับ New password และ Confirm new password; อย่าวางหรือบันทึกข้อมูลลับ:

```

npm run users -- create --username uat-assistant --display-name Assistant --role assistant
npm run users -- create --username uat-doctor --display-name Doctor --role doctor

```

### ผลที่มองเห็น

แต่ละคำสั่งจบด้วย User account command completed และมีเพียงสองบัญชีนี้

### ถ้าไม่ตรงให้หยุด

เมื่อ provisioning failure ให้ BLOCKED; ห้ามสร้างบัญชีอื่นหรือรัน create command ซ้ำ.

## Checkpoint 10 - เปิดระบบและเข้าสู่ระบบสองบทบาท

### เป้าหมาย

เปิด host เฉพาะ loopback, ตรวจสอบ health และยืนยัน workspace ตาม role

### ทำตามนี้

ให้หน้าต่างนี้เป็น host PowerShell แล้วรัน:

```
$env:CAREFLOW_HOST = '127.0.0.1'
$env:CAREFLOW_PORT = '3001'
$env:CAREFLOW_COOKIE_SECURE = 'false'
npm start
```

เมื่อ host ทำงาน เปิด PowerShell ภายในหน้าต่างที่สองและรัน:

```
Invoke-RestMethod -Uri 'http://127.0.0.1:3001/api/health' -Method Get
```

### เข้าสู่ระบบครั้งแรก

1. ให้หน้าต่าง host PowerShell ที่กำลังรัน npm start เปิดอยู่
2. เปิด Edge หรือ Chrome สอง Browser Profile ที่แยกกัน
3. ในแต่ละ Browser Profile ให้เปิด http://127.0.0.1:3001
4. ลงชื่อเข้าใช้ครั้งแรกด้วย uat-assistant และครั้งแรกด้วย uat-doctor โดยไม่บันทึกข้อมูลกลับ
5. ยอมรับ Pilot acknowledgement และเปลี่ยน initial password ผ่าน first-login UI ที่แสดง
6. ยืนยันว่า Assistant ไปยัง Assistant workspace และ Doctor ไปยัง Doctor workspace ที่ถูกต้อง

### ผลที่ต้องเห็น

health check สำเร็จ และ browser เปิดเฉพาะ http://127.0.0.1:3001

### ถ้าไม่ตรงให้หยุด

หาก health check หรือ first login ไม่ตรง ให้ BLOCKED; อย่าเปิด port สู่เครือข่าย เก็บฐานข้อมูลเดิมไว้ และกลับไปใช้ Administrator runbook เท่านั้น.

## หยุด CareFlow อย่างปลอดภัย

กลับไปทีหน้าต่าง PowerShell ที่กำลังรัน npm start แล้วกด Ctrl+C หนึ่งครั้ง รอจน prompt PS ...> กลับมา ห้ามลบไฟล์ .sqlite, -wal, -shm หรือโฟลเดอร์ lock เพื่อบังคับหยุดระบบ

## ขั้นถัดไป: เริ่ม UAT

เมื่อทั้งสองบัญชีเข้าสู่ระบบได้ ให้ใช้ฐานข้อมูลเดิมและทำตาม [คู่มือ UAT Journey ทั้ง 5 Scenario](#) พร้อมบันทึกผลใน [UAT checklist](#) หากต้อง restart, ตรวจสอบ session expiry หรือแก้อสถานะ BLOCKED ให้กลับไปใช้ [Administrator runbook](#) เท่านั้น